App Dev Project Report

1. Student Details

Name: Mrinal Pandey
Roll Number: 23f2000333
Email: 23f2000333@ds.study.iitm.ac.in
About Me: I am a student pursuing the IIT Madras BS Degree program.

---

2. Project Details

Project Title: Hospital Management System

Problem Statement:

To build a Hospital Management System (HMS) web application that allows Admins, Doctors, and Patients to interact with the system based on their roles.

Approach:

The system is developed using Flask with role-based dashboards. Each user role performs actions suited to their responsibilities. The project follows a simple MVC-style structure with SQLite as the backend database created programmatically.

---

3. AI/LLM Declaration

I used ChatGPT (GPT-5) to assist in writing SQLAlchemy model definitions, creating API documentation samples, and improving variable naming consistency.
The extent of AI/LLM usage is around 10–15%, limited to code suggestions and documentation formatting.
All final implementation logic, debugging, and integration were done manually.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
Flask-Login	User authentication & session management
Flask-SQLAlchemy	ORM for SQLite database
SQLite	Application database, created programmatically
Jinja2	HTML templating
Bootstrap 5	Styling & responsive layout
HTML, CSS	Front-end interface
Werkzeug	Password hashing

---

5. Database Schema / ER Diagram

Tables:

1. **User** — id, username, email, password_hash, role, is_active
2. **PatientProfile** — id, user_id, full_name, age, gender, phone, address, is_blacklisted
3. **DoctorProfile** — id, user_id, full_name, specialization, department_id, years_experience, bio, is_active, is_blacklisted
4. **Department** — id, name, description
5. **Appointment** — id, patient_id, doctor_id, date, time, status, created_at, reason
6. **TreatmentRecord** — id, appointment_id, diagnosis, prescription, notes
7. **DoctorAvailability** — id, doctor_id, date, start_time, end_time, is_booked

Relationships:

- One-to-One->User->PatientProfile
- One-to-One->User->DoctorProfile
- One-to-Many->Department->DoctorProfile
- One-to-Many->PatientProfile->Appointment
- One-to-Many->DoctorProfile->Appointment
- One-to-One->Appointment->TreatmentRecord
- One-to-Many->DoctorProfile->DoctorAvailability

---

6. API Resource Endpoints

Endpoint	Method	Description
`/api/doctors`	GET	Returns list of doctors
`/api/appointments/<id>`	GET	Return appointment details in JSON format
`/api/patient/history`	GET	Return appointment and treatment history

---

7. Architecture and Features (optional)

Architecture Overview:

- **app.py -** main application file
- **/templates -** Jinja2 HTML pages
- **/static -** CSS and bootstrap assets
- **/hospital.db** - SQLite DB generated programmatically

Implemented Features:

- Patient registration and login
- Secure password hashing and user session handling
- Admin Dashboard: view counts, search, manage doctors & patients, and view appointments.
- Doctor Dashboard: upcoming/past appointments, availability, treatment records, patient history.
- Patient Dashboard: search doctors, booking, cancelling, treatment history, profile edit.
- Appointment status transition (Booked -> Completed -> Cancelled).
- Treatment history storage and visibility for doctor and patient.
- Blacklist enforcement for doctor & patient access.
- Next 7 days availability enforcement.
- Search functionality for doctors and patients.

Additional Features:

- Doctor & patient search
- Blacklisting mechanism
- Patient treatment history view
- Server-side validation

---

8. Video Presentation

Drive Link:

https://drive.google.com/file/d/1lHEeXZ8cpD5v95hSZUNHfk03bGdQYPrX/view?usp=sharing